

Design of Information Structures (CS126)

Programming Assignment

Feedback for 5679421

Mark: 90%

Grade: 1

Feedback for all students on this assignment

Generic Feedback: This was a strong cohort result overall. The vast majority of you passed all unit tests, and the benchmarking data for most submissions shows evidence of serious engagement with the core principle of the coursework: precomputing and maintaining data structures at insertion/deletion so that query methods can return results efficiently. The key takeaway from this assignment is the distinction between storing data and indexing it. Storing a list of ratings lets you retrieve a specific entry efficiently, but answering a query like 'what is the average rating for film X?' efficiently requires a separate structure maintained in parallel.

A number of submissions used Java's inbuilt data-structures or the provided `MyArrayList` rather than self-implemented ones. These submissions were penalised (see the specification).

Marking process: Each submission was marked by one of the markers, across the four criteria pointed out in the coursework lecture (correctness, efficiency, code analysis, report).

The automated stage ran your unit tests and benchmarked the runtime performance of your methods, classifying each into a fast/ok/slow band based on timing data. The difficulty of the task was also factored in (3 tiers of difficulty). A few submissions had compilation or runtime errors that resulted in 0 marks for the automated part of the marking and for these, only the next two components were considered.

The code analysis component itself had two parts. The first was Algorithmic Optimality (whether your data structures and query methods were efficient, the presence of auxiliary index structures maintained at `add()` time, the absence of unnecessary $O(n)$ scans in query methods, etc). Using `java.util` data structures or `MyArrayList` resulted in a score of zero for this part. The second was Code Quality, which took into account aspects such as naming (are field, method, and class names descriptive and consistent across files?); comments (are non-obvious decisions explained? not just what the code does, but why?); and modularity (is each data structure in its own file in the structures package, are methods a reasonable length, and is there a reusable implementation rather than duplicated logic?).

The final component was the report. This had three subcomponents. First, "Consistency" assessed whether your report accurately described what your code actually does. Markers checked that all three store classes (`Movies`, `Ratings`, `Credits`) were named explicitly, that specific structural claims could be verified in the code, and looked for discrepancies between what the report claimed and what the code contained. Vague or generic descriptions that could not be mapped to specific fields or files scored lower. Second, "Justification" assessed the quality of reasoning in your report. Markers were looking for explanations of why each data structure was chosen and not just what it is. Strong submissions compared at least one realistic alternative (e.g., 'I chose a `HashMap` over a sorted array because lookups are by ID and insertions are frequent'), based their reasoning on the specific access patterns of this coursework, and acknowledged relevant tradeoffs such as memory usage versus query speed.

Finally, “Complexity Analysis” assessed whether your report used Big-O notation correctly and specifically. Markers checked that complexity claims were applied to your own operations rather than stated generically (e.g., 'HashMap is $O(1)$ ' without further context), that the claims were correct relative to the actual code, that sorting complexities were addressed, and that all parameters in complexity expressions were defined or clear from the context (e.g., what n or k represents in your implementation). Incorrect claims (e.g., stating $O(1)$ for a method that contains a scan) reduced scores even when Big-O notation was otherwise used throughout.

Hope this helps and let me know if you have any questions.

Feedback on your submission

Separate chaining justification for sparse IDs, bit-shifted long for composite Ratings keys, insertion sort for small cast lists, and six reverse indexes in Credits are all good choices. Areas for Improvement:

The report could have had more thorough complexity analysis justifying some of the assertions about complexities.